SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 — CONCEPT MAPPING

Course Code: CSA0305
Course Title: Data Structures
CO Assessed: CO1 – Imitation (BL3, BL4)
Weightage: 35% of CIA
Total Marks: 100
Date: 10/07/2026

CO1 Statement: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.*

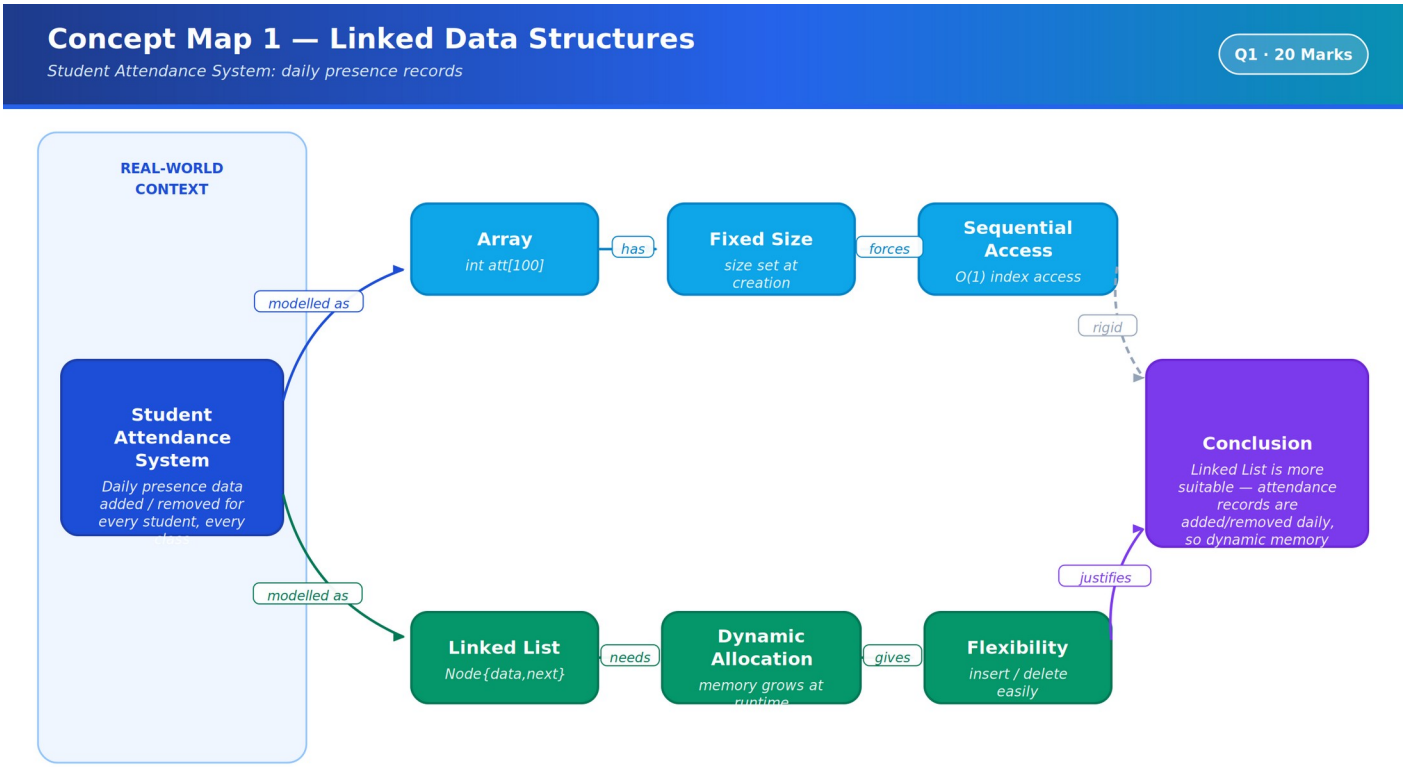
Student Name: Jaswanth.S
Reg. No.: 192525214
Section / Batch: AIML

Summary of Questions

Q.No	Topic Focus	Section	Marks
1	Array → Fixed Size → Sequential Access → Linked List → Dynamic Allocation → Flexibility	Linked Data Structure	20
2	Primitive → Non-Primitive → Linear → Non-Linear Structures → Applications	Linear & Non-Linear DS	20
3	Searching → Linear Search → Binary Search → Sorted Data → Time Complexity	Search Data Structure	20
4	Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency	Recursion, Performance	20
5	Tree → Nodes → Levels → Traversal (DFS/BFS) → Searching → Efficiency	Tree Data Structures	20

Question 1 — Linked Data Structures (20 Marks)

A student attendance system records daily presence data for every student, in every class, every day — meaning records are constantly added and never stay a fixed size. The concept map below links Array → Fixed Size → Sequential Access against Linked List → Dynamic Allocation → Flexibility to reason about which structure fits this workload.



Justification: Sequential access in arrays makes insert/delete costly (O(n) shifting); linked lists give O(1) insert/delete once positioned — ideal for daily updates.

Understanding of Concepts

An Array stores attendance values in contiguous memory with a size fixed at declaration time, so it must be accessed sequentially by index. A Linked List, in contrast, stores each record as a node holding data and a pointer to the next node, letting memory be allocated dynamically as students are added or removed.

Connections Between Ideas

- **Array → Fixed Size:** a fixed block of contiguous memory decided at creation time, which forces
- **Fixed Size → Sequential Access:** locating any record means walking through the array in index order
- **Linked List → Dynamic Allocation:** each attendance node can be allocated only when a new student record is created
- **Dynamic Allocation → Flexibility:** because memory grows and shrinks on demand, records can be inserted or deleted anywhere without shifting other data

Logical Structure & Justification

Since daily attendance changes constantly (new admissions, transfers, absentee corrections), a Linked List is the more suitable structure: its dynamic allocation avoids wasting or running out of fixed space, and its flexibility keeps insert/delete operations efficient (O(1) once positioned) compared to an array's costly O(n) shifting.

Clarity of Representation

The map places the two structures as parallel, colour-coded chains (blue for Array, green for Linked List) that both originate from the real-world problem and converge on a single evidence-based conclusion, making the comparison and final justification easy to follow at a glance.

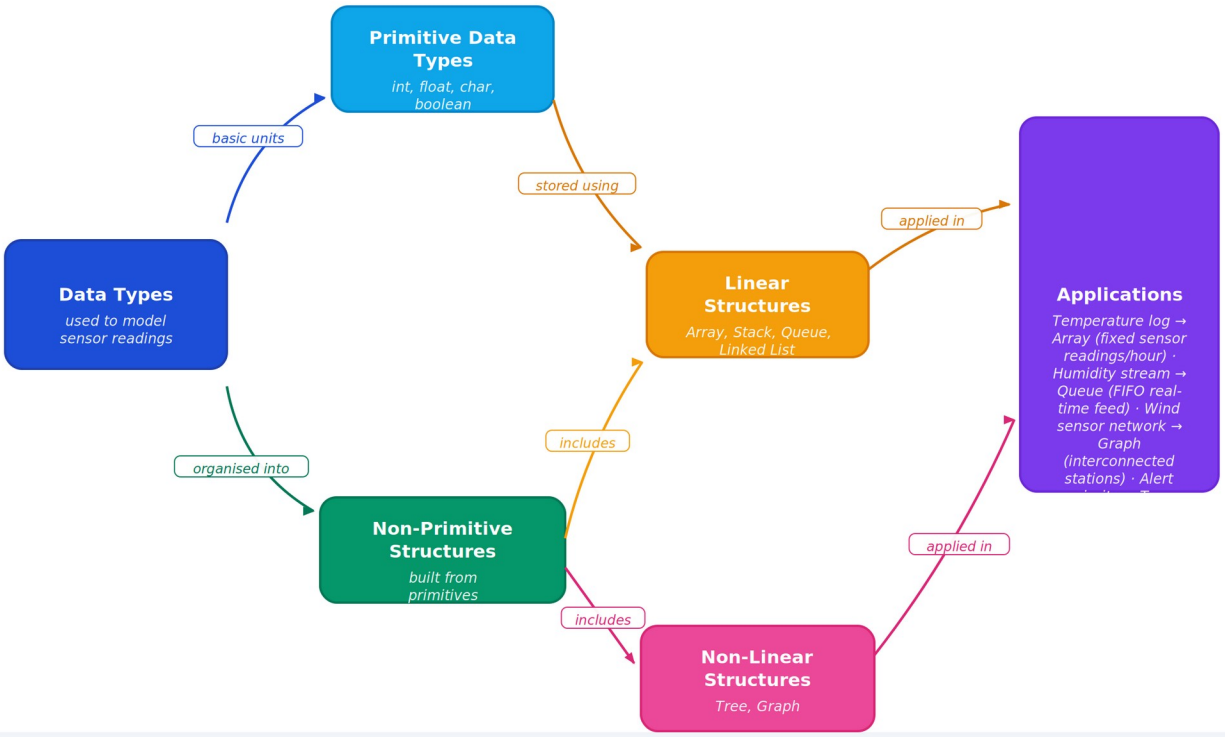
Question 2 — Linear & Non-Linear Data Structures (20 Marks)

A weather monitoring system continuously collects temperature, humidity, and wind data. The concept map traces how raw Primitive Data Types combine into Non-Primitive Structures, which split into Linear and Non-Linear Structures, and finally map onto real sensor Applications.

Concept Map 2 — Data Structure Classification

Weather Monitoring System: continuous temperature, humidity & wind data

Q2 · 20 Marks



Continuous sensor data streams naturally onto Linear structures (time-ordered readings) while station relationships and alert hierarchies fit Non-Linear structures.

Understanding of Concepts

Primitive Data Types (int, float, char, boolean) are the language's basic building blocks and directly hold single sensor readings such as one temperature value. Non-Primitive Structures are built by composing primitives — arrays, records, or pointers — to hold many readings together.

Connections Between Ideas

- **Primitive → Non-Primitive:** individual readings are the basic units combined to build
- **Non-Primitive → Linear / Non-Linear:** Non-Primitive Structures split by how elements relate to each other
- **Linear Structures:** elements are arranged sequentially — used for time-ordered sensor logs
- **Non-Linear Structures:** elements are arranged hierarchically or by relationship — used for sensor networks and alert priority

Logical Structure & Justification

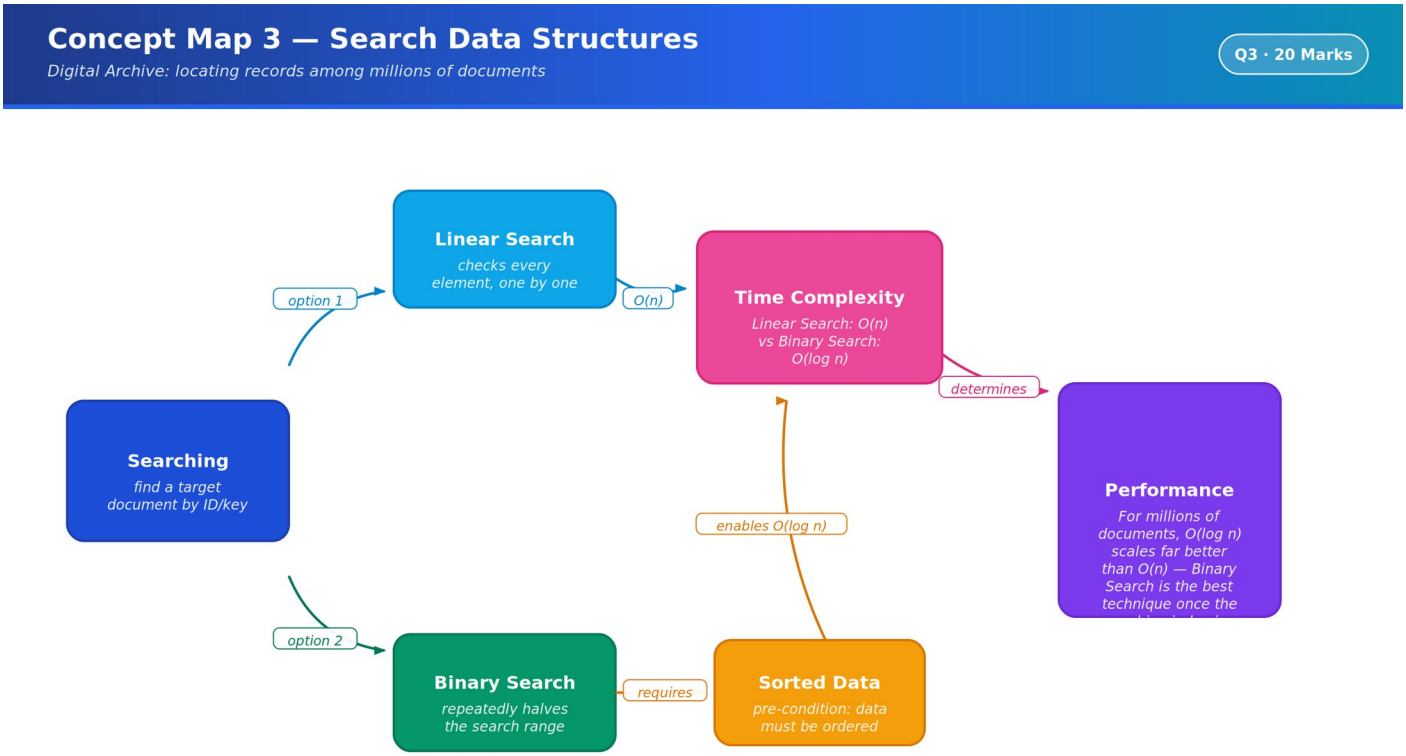
Continuous data processing benefits from both branches: an Array or Queue (Linear) naturally models a time-ordered stream of temperature or humidity readings, using FIFO order to process the latest data first, while a Graph or Tree (Non-Linear) models relationships between multiple wind sensor stations or represents escalating alert severity levels.

Clarity of Representation

The map fans out left to right — from single data types, through the Linear/Non-Linear split, to concrete applications — so a reader can trace exactly which structure serves which part of the weather system without ambiguity.

Question 3 — Search Data Structures (20 Marks)

A digital archive stores millions of documents and must locate a specific one quickly. The concept map links Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance to justify the best retrieval technique.



Data organization drives efficiency: an unsorted archive forces Linear Search (slow at scale); sorting once unlocks fast Binary Search for every future query.

Understanding of Concepts

Linear Search checks each document one after another until a match is found, requiring no particular ordering. Binary Search instead repeatedly halves the search range, but only works correctly when the underlying data is Sorted.

Connections Between Ideas

- **Linear Search → Time Complexity:** scanning the archive one record at a time takes O(n) time in the worst case
- **Sorted Data → Binary Search:** a sorted index lets each comparison eliminate half the remaining documents
- **Binary Search → Time Complexity:** this halving gives Binary Search its O(log n) time complexity
- **Time Complexity → Performance:** lower time complexity directly translates into faster real-world Performance

Logical Structure & Justification

With millions of documents, the gap between O(n) and O(log n) is enormous — Linear Search could mean scanning millions of entries, while Binary Search narrows the same search to roughly 20 comparisons. The one-time cost of keeping the archive index sorted is repaid on every subsequent search, making Binary Search the best technique for this scale.

Clarity of Representation

The two search techniques are shown as competing branches that reconverge at Time Complexity and finally at a single Performance verdict, visually reinforcing why data organisation (sorting) is the deciding factor in search efficiency.

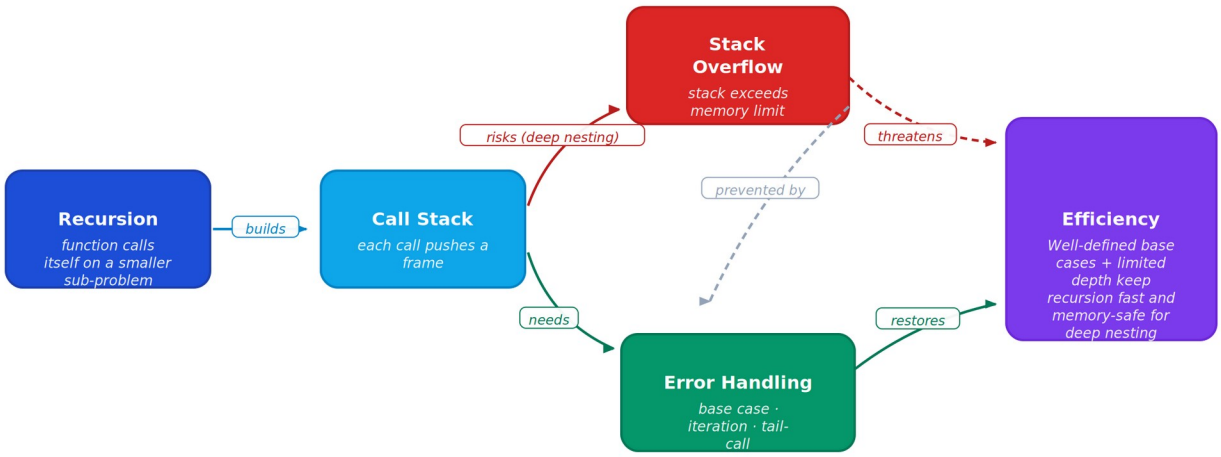
Question 4 — Recursion & Performance Analysis (20 Marks)

A program processes nested expressions using recursion and may encounter deep nesting. The concept map links Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency to analyse the risk and its mitigation.

Concept Map 4 — Recursion & Performance

Program evaluating deeply nested expressions

Q4 · 20 Marks



Deep nesting can exhaust the call stack; robust programs pair recursion with a solid base case, depth limits, or an iterative/tail-call rewrite to protect efficiency.

Understanding of Concepts

Recursion solves a problem by having a function call itself on a smaller sub-problem. Every call is pushed onto the Call Stack as a stack frame holding its local variables and return address, and is only removed once that call completes.

Connections Between Ideas

- **Recursion → Call Stack:** each recursive call adds a new frame to the call stack
- **Call Stack → Stack Overflow:** very deep nesting keeps pushing frames until available memory is exhausted
- **Stack Overflow → Error Handling:** a well-defined base case, depth limits, or converting to iteration/tail-calls prevents this failure
- **Error Handling → Efficiency:** preventing overflow while keeping calls minimal preserves both correctness and speed

Logical Structure & Justification

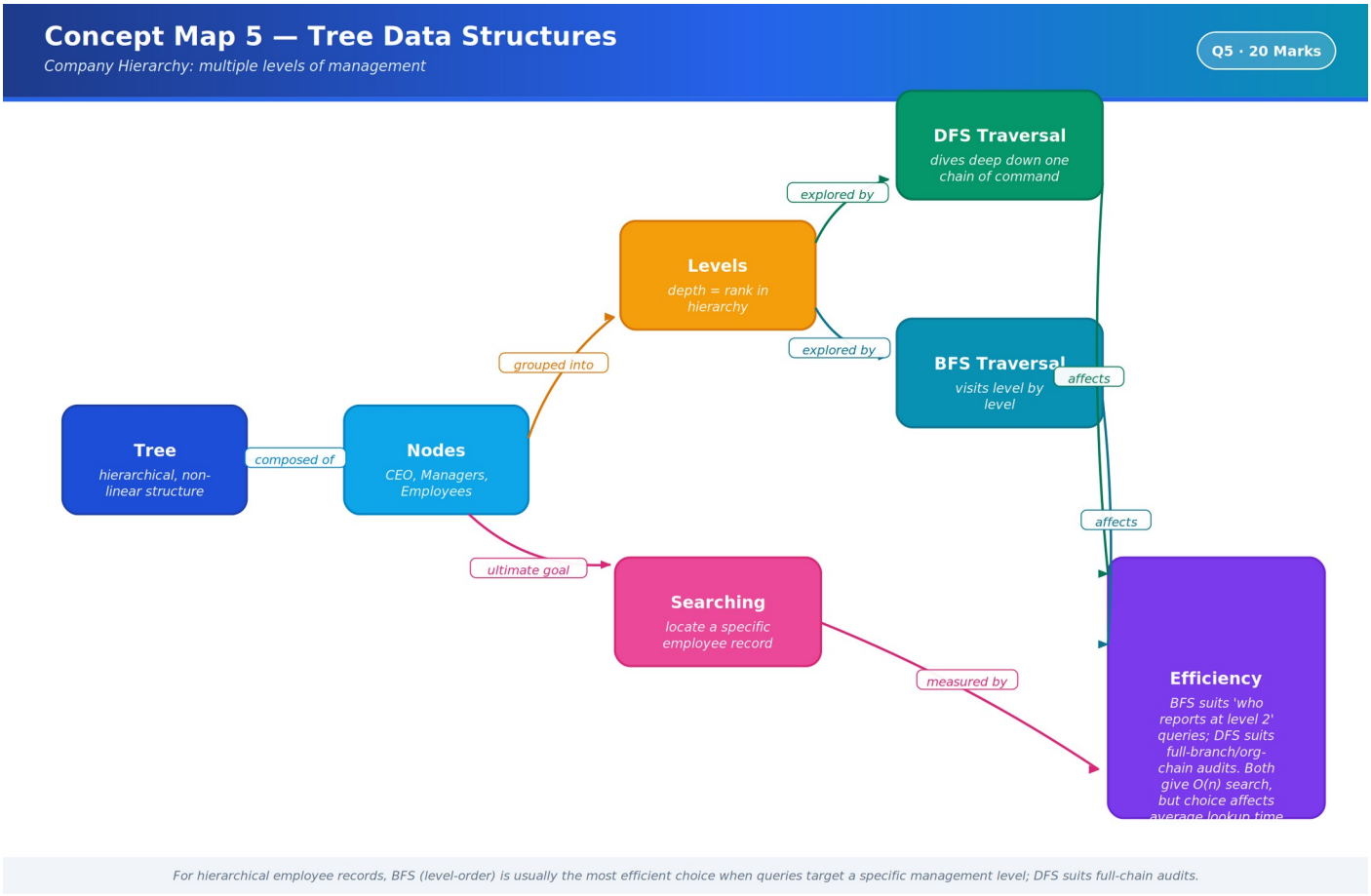
For deeply nested expressions, the safest strategy is a combination approach: enforce a solid base case so recursion terminates as early as possible, add an explicit depth check or convert the routine to an iterative loop for very deep inputs, and use memoization to avoid repeating identical sub-calls — together these keep the program both memory-safe and efficient.

Clarity of Representation

A single linear chain highlights the cause-and-effect relationship from Recursion to potential failure (Stack Overflow, shown in red) and back to a safe, efficient outcome, so the risk and its resolution are both visible in one glance.

Question 5 — Tree Data Structures (20 Marks)

A company maintains hierarchical employee records with multiple levels of management. The concept map connects Tree → Nodes → Levels → Traversal (DFS/BFS) → Searching → Efficiency to identify the best way to retrieve employee data.



For hierarchical employee records, BFS (level-order) is usually the most efficient choice when queries target a specific management level; DFS suits full-chain audits.

Understanding of Concepts

A Tree is a non-linear, hierarchical structure whose Nodes represent entities such as the CEO, managers, and employees. Every node sits at a Level that reflects its rank/depth in the reporting hierarchy — the CEO at level 0, direct reports at level 1, and so on.

Connections Between Ideas

- **Tree → Nodes:** individual employee records are composed together into the hierarchy
- **Nodes → Levels:** nodes are grouped by their depth, forming the organisation's reporting levels
- **Levels → Traversal (DFS / BFS):** Depth-First Search dives down one management chain before backtracking; Breadth-First Search visits every node at one level before moving deeper
- **Traversal → Searching:** either traversal can be used to locate a specific employee record

Logical Structure & Justification

The right traversal depends on the query: BFS (level-order) is the most efficient choice for questions like 'who are all the managers at level 2', since it explores level-by-level and can stop as soon as the target level is exhausted. DFS is better suited to full-chain audits, such as tracing one employee's entire reporting line to the CEO. Both traversals give O(n) worst-case search time, but the right choice minimises the average number of nodes visited.

Clarity of Representation

The map branches from a single hierarchy into the two traversal strategies before reconverging on Searching and Efficiency, making the trade-off between DFS and BFS explicit and easy for a reader to evaluate for any given query.